

黑白棋-实训报告

2312966 林晖鹏

一、问题重述

黑白棋问题：

黑白棋 (Reversi)，又称翻转棋，是一个经典的策略性游戏。一般棋子双面为黑白两色，故称“黑白棋”。因为行棋之时将对方棋子翻转，则变为己方棋子，故又称“翻转棋” (Reversi)。黑白棋使用 8x8 大小的棋盘，由两人执黑子和白子轮流下棋，最后子多方为胜方。

游戏规则：

1. 黑方先行，双方交替下棋。
2. 一步合法的棋步包括：
 - 在一个空格处落下一个棋子，并且翻转对手一个或多个棋子；
 - 新落下的棋子必须落在可夹住对方棋子的位置上，对方被夹住的所有棋子都要翻转过来，可以是横着夹，竖着夹，或是斜着夹。夹住的位置上必须全部是对手的棋子，不能有空格；
 - 一步棋可以在数个（横向，纵向，对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不翻某个棋子。
3. 如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。
4. 如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。
5. 棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。
6. 如果某一方落子时间超过 1 分钟 或者 连续落子 3 次 不合法，则判该方失败。

作业要求：

- 使用蒙特卡洛搜索树算法求解某一棋盘下的较优解，从而实现一个AI选手。
- 使用python语言实现代码。

题目解读：

题目已经构建好框架了，我们只需要在某一棋盘下有限时间内通过蒙特卡洛搜索树得到一个较优解即可。而在搜索过程中，我们要模拟黑白双方交替下棋（零和问题），此时搜索树还是个MinMax问

题，我们要模拟双方作出该方的最优解。

二、设计思想

蒙特卡洛搜索树主要分为四个步骤：选择、扩展、模拟、反向传播。

1. 选择

从根节点开始，如果子节点都已经被扩展过了，则选择UCB值最大的一个子节点继续往下选择，直到选择到出现子节点未被扩展，则扩展子节点；或者到达叶节点，则直接模拟后反向传播。

2. 扩展

选择一个未被扩展的节点，从这个节点往下进行模拟，得到新的UCB值之后作为这个节点的UCB值。

3. 模拟

用某种策略（例如：随机游走、贪心策略等）不断选择子节点进行向下搜索，直到到达叶节点，返回叶节点的收益，作为扩展节点的UCB值。

4. 反向传播

将根节点到扩展节点路径上的所有节点的收益加上模拟返回的收益值，并且访问次数都加1。（更新UCB值）

(1) 节点类

★ 由于是MinMax问题，选择时参考的UCB值不一样，而每个节点反向传播以及记录的收益都是根节点颜色的总收益。所以，当节点和根节点颜色相同时，UCB值采用「总收益/访问次数 + C*次数因子」；当节点和根节点颜色不同时，UCB值采用「64 - 总收益/访问次数 + C*次数因子」（表示另一方的平均收益）。

所以类里面设置了两个UCB函数

这里我们的 **C** 设置为 2。

函数主要有：

- `UCBX`：返回「总收益/访问次数 + C*次数因子」。
- `UCBO`：返回「64 - 总收益/访问次数 + C*次数因子」。
- `avar`：返回「总收益/访问次数」，用于最终的选择，希望选择平均收益最高的，而不需要考虑探索次数的影响。

代码块

```
1  class Node:
2      def __init__(self, board, color, action=None, parent=None):
3          """
4          节点初始化
5          :param board: 目前棋盘
6          :param color: 节点颜色
7          :param action: 合法节点落子位置
8          :param value: 节点收益，一直记录的是黑棋的棋子数
9          :param times: 节点访问次数
10         """
11         self.board = copy.deepcopy(board)
12         self.color = color
13         self.action = action
14         self.children_action = list(self.board.get_legal_actions(self.color))
15         self.children = []
16         self.value = 0
17         self.times = 0
18         self.parent = parent
19
20         #获得子节点的可选行动
21     def get_childern_action(self):
22         return self.children_action
23
24     def get_childern(self):
25         return self.children
26
27     def UCBX(self, sumtimes):
28         """
29         计算 UCB 值，这里是黑棋选择节点的时候的UCB
30         这里假设C为1
31         """
32         C = 2
33         if self.times == 0: #如果自己没有被访问过
34             return None
35         return self.value / self.times + C * ((2 * sumtimes / self.times) **
0.5)
36     def UCBO(self, sumtimes):
37         """
38         计算 UCB 值，这里是白起选择节点时候考虑的UCB
39         这里假设C为1
40         """
41         C = 2
42         if self.times == 0: #如果自己没有被访问过
43             return None
```

```

44         return 64 - self.value / self.times + C * ((2 * sumtimes / self.times)
45             ** 0.5)
46
47     def avar(self):
48         return self.value / self.times if self.times != 0 else 0

```

(2) 选择和扩展

由于扩展代码比较简单，我们将其合并到选择里面。

代码块

```

1  def choose(self, node):
2      '''
3      到node下面选择节点
4      如果没有孩子了，就返回自己
5      如果self的孩子节点没有被访问过，则选择一个孩子节点进行扩展
6      返回要被模拟的节点，就是被新扩展的那个节点
7      '''
8      # 游戏结束
9      b_list = list(node.board.get_legal_actions('X'))
10     w_list = list(node.board.get_legal_actions('O'))
11     if len(b_list) == 0 and len(w_list) == 0:
12         return node
13
14
15     # 如果没有孩子了，需要弃权
16     if len(node.get_children_action()) == 0:
17         return self.choose(Node(node.board, self.other_color(node.color),
18     None, node))
19
20     children_action = node.get_children_action()
21     children_has_act = [child.action for child in node.get_children()]
22     missing_acts = [act for act in children_action if act not in
23     children_has_act]
24
25     if len(missing_acts) != 0: # 如果还有孩子没有被访问过
26         # 选择一个扩展
27         act = missing_acts[0]
28         new_board = copy.deepcopy(node.board)
29         new_board._move(act, node.color)
30         node.children.append(Node(new_board, self.other_color(node.color),
31     act, node))
32
33     # 返回被扩展的节点
34     return node.children[-1]

```

```

31
32     # 如果所有的孩子节点都被访问过了
33     Max_UCB = -1
34     child_ = None
35     if node.color == self.color: #如果是黑棋，则选择
36         for child in node.children:
37             # 计算UCB值
38             ucb = child.UCBX(node.times)
39             if ucb > Max_UCB:
40                 Max_UCB = ucb
41                 child_ = child
42         return self.choose(child_)
43     else:
44         for child in node.children:
45             # 计算UCB值
46             ucb = child.UCB0(node.times)
47             if ucb > Max_UCB:
48                 Max_UCB = ucb
49                 child_ = child
50         return self.choose(child_)

```

步骤：

1. 先判断游戏有没有结束。
2. 如果没有合法落子位置，游戏又没结束，我们应该交换颜色，给另一方下棋，继续选择。
3. 我们从孩子节点中获取孩子节点的 `children_acts`，然后从当前棋盘获取合法落子位置 `acts`。
4. 对照两个acts，如果存在某个act在 `children_acts` 没有出现，则说明有孩子未被扩展，则选择扩展最左边的孩子，返回这个孩子。
5. 如果孩子都被扩展过了，就遍历每一个孩子的UCB值（这里UCB值的形式取决于节点颜色和根节点颜色是否一致，前文有解释），找到UCB值最大的孩子节点继续进行选择。

(3) 模拟

我们模拟的策略可以选择随机游走或者贪心算法（选择翻转对方最多的棋子）。

除此之外，在研究高级对局的时候，发现，我们还可以优先下边上的棋子，再结合上面两个策略，可以和高级打的时候更高分。

贪心策略：

代码块

```

1  def simulation(self, node):

```

```

2      """
3      模拟
4      node是扩展的节点,要在他的子节点上进行模拟,返回收益,黑子棋数 board.count(color)
5      :param node: 当前节点
6      """
7      # 将要走的位置
8      action = None
9      # 进行模拟
10     children_action = node.get_children_action()
11
12     # 如果此时游戏结束了,就返回黑棋收益
13     b_list = list(node.board.get_legal_actions('X'))
14     w_list = list(node.board.get_legal_actions('O'))
15     is_over = len(b_list) == 0 and len(w_list) == 0 # 返回值 True/False
16     if is_over:
17         return node.board.count(self.color)
18
19     # 如果没有合法位置下了,就弃权,让对方下
20     if len(children_action) == 0:
21         return self.simulation(Node(node.board, self.other_color(node.color),
22                                     None, node))
23
24     # 如果有子节点,则贪心选择一个子节点进行模拟
25     max_num = 0
26     for act in children_action:
27         # 进行模拟
28         new_board = copy.deepcopy(node.board)
29         # 模拟下棋并计算反转的个数
30         num = new_board._move(act, node.color)
31         if isinstance(num, bool):
32             continue
33         num = len(num)
34         if num > max_num:
35             max_num = num
36             action = act
37
38     if action is None:
39         # 返回这个子节点上面黑棋的棋子数
40         return node.board.count(self.color)
41
42     node.board._move(action, node.color)
43
44     nextnode = Node(node.board, self.other_color(node.color), act)
45     return self.simulation(nextnode)

```

随机策略:

代码块

```
1  def simulation(self, node):
2      children_action = node.get_children_action()
3
4      # 当前局面结束的判断：用当前节点状态而非整棵树
5      b_list = list(node.board.get_legal_actions('X'))
6      w_list = list(node.board.get_legal_actions('O'))
7      if len(b_list) == 0 and len(w_list) == 0:
8          return node.board.count(self.color)
9
10     # 如果没有合法走法，则切换对方下
11     if len(children_action) == 0:
12         return self.simulation(Node(node.board,
self.other_color(node.color), None, node))
13
14     # 改为随机模拟
15     action = random.choice(children_action)
16     node.board._move(action, node.color)
17
18     nextnode = Node(node.board, self.other_color(node.color), action)
19     return self.simulation(nextnode)
```

步骤：

1. 先判断是否游戏结束。
2. 如果没有合法位置，则弃权。
3. （贪心策略）遍历每个合法行为，复制模拟棋盘进行下棋，然后返回每一个行为的翻转棋子个数，选出翻转最多的行为。
4. （随机策略）随机选取一个合法行为。
5. 根据选取的行为创建新的节点继续进行模拟。

（4）反向传播

节点里面存储了父亲节点，反向传播模拟收益，并且增加一次访问次数，知道传播至根节点。

代码块

```
1  def backing(self, node, value):
2      node.value += value
3      node.times += 1
4      if node.parent is None: # 说明到根节点了
5          return
```

```
6         else:
7             self.backing(node.parent, value)
```

(5) 蒙特卡洛搜索

由于每次使用蒙特卡洛树必须在60s以内，我们这里设置每一步的下棋不断搜索蒙特卡洛树20s，设置C为2，以便于得到一个较优的解。

如果本身没有合法位置，就返回None。

代码块

```
1  def oneSearch(self):
2      #先找节点扩展
3      node = self.choose(self.root)
4      # 进行模拟
5      value = self.simulation(node)
6      # 进行回传
7      self.backing(node, value)
8
9  def Search(self):
10     # 记录开始时间
11     start_time = time.time()
12
13     # 搜索20秒或者直到时间到
14     while time.time() - start_time < 20:
15         # 进行一次搜索
16         self.oneSearch()
17
18     # 计算UCB值
19     Max_UCB = -1
20     action = None
21     for child in self.root.children:
22         ucb = child.avar()
23         if ucb > Max_UCB:
24             Max_UCB = ucb
25             action = child.action
26     return action
27
```

三、代码内容

完整代码

代码块

```
1  import copy
2  import random
3  import time
4  class Node:
5      def __init__(self, board, color, action=None, parent=None):
6          """
7          节点初始化
8          :param board: 目前棋盘
9          :param color: 节点颜色
10         :param action: 合法节点落子位置
11         :param value: 节点收益，一直记录的是黑棋的棋子数
12         :param times: 节点访问次数
13         """
14         self.board = copy.deepcopy(board)
15         self.color = color
16         self.action = action
17         self.children_action = list(self.board.get_legal_actions(self.color))
18         self.children = []
19         self.value = 0
20         self.times = 0
21         self.parent = parent
22
23         #获得子节点的可选行动
24     def get_children_action(self):
25         return self.children_action
26
27     def get_children(self):
28         return self.children
29
30     def UCBX(self, sumtimes):
31         """
32         计算 UCB 值，这里是黑棋选择节点的时候的UCB
33         这里假设C为1
34         """
35         C = 2
36         if self.times == 0: #如果自己没有被访问过
37             return None
38         return self.value / self.times + C * ((2 * sumtimes / self.times) **
0.5)
39     def UCBO(self, sumtimes):
40         """
41         计算 UCB 值，这里是白起选择节点时候考虑的UCB
42         这里假设C为1
```

```

43         '''
44         C = 2
45         if self.times == 0: #如果自己没有被访问过
46             return None
47         return 64 - self.value / self.times + C * ((2 * sumtimes / self.times)
48             ** 0.5)
49
50     def avar(self):
51         return self.value / self.times if self.times != 0 else 0
52
53 class UTCSearch:
54     def __init__(self, board, color):
55         """
56         初始化
57         :param board: 当前棋盘
58         :param color: 当前颜色
59         """
60         self.board = copy.deepcopy(board)
61         self.color = color
62         self.root = Node(board, color)
63         # 进行第一次初始化
64
65     def other_color(self, color):
66         if color == 'X':
67             return 'O'
68         else:
69             return 'X'
70
71     # def simulation(self, node):
72     #     """
73     #     模拟
74     #     node是扩展的节点,要在他的子节点上进行模拟,返回收益,黑子棋数
75     board.count(color)
76     #     :param node: 当前节点
77     #     """
78     #     # 将要走的位置
79     #     action = None
80     #     # 进行模拟
81     #     children_action = node.get_children_action()
82
83     #     # 如果此时游戏结束了,就返回黑棋收益
84     #     b_list = list(node.board.get_legal_actions('X'))
85     #     w_list = list(node.board.get_legal_actions('O'))
86     #     is_over = len(b_list) == 0 and len(w_list) == 0 # 返回值 True/False

```

四、实验结果

平台测试

每个难度，先后手黑白棋均通过，下面是结果截图（这里领先数由于用的是随机策略，不一定每次结果相同）。

初级+黑棋先手：

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	321s	黑棋获胜, 领先棋子数: 50

确定

测试详情 隐藏棋盘

A B C D E F G H

1

2

3

4

5

6

7

8

棋局胜负: 黑棋赢

先后手: 黑棋先手

棋局难度: 初级

当前棋子: 白棋

当前坐标: E5

4 / 64

确定

初级+白棋后手：

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	621s	白棋获胜, 领先棋子数: 32

确定

测试详情 隐藏棋盘

A B C D E F G H

1

2

3

4

5

6

7

8

棋局胜负: 白棋赢

先后手: 白棋后手

棋局难度: 初级

当前棋子: 白棋

当前坐标: E5

4 / 64

确定

中级+黑棋先手：

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	324s	黑棋获胜, 领先棋子数: 28

确定

测试详情 隐藏棋盘

A B C D E F G H

1

2

3

4

5

6

7

8

棋局胜负: 黑棋赢

先后手: 黑棋先手

棋局难度: 中级

当前棋子: 白棋

当前坐标: E5

4 / 64

确定

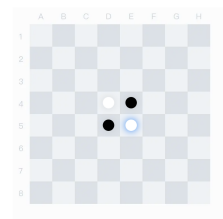
中级+白棋后手：

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	626s	白棋获胜, 领先棋子数: 18

确定

测试详情 隐藏棋盘



棋局胜负: 白棋赢
先后手: 白棋后手
棋局难度: 中低
当前棋子: 白棋
当前坐标: E5

确定

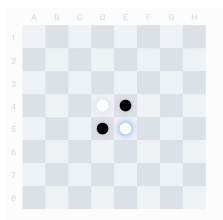
高级+黑棋先手:

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	346s	黑棋获胜, 领先棋子数: 18

确定

测试详情 隐藏棋盘



棋局胜负: 黑棋赢
先后手: 黑棋先手
棋局难度: 高底
当前棋子: 白棋
当前坐标: E5

确定

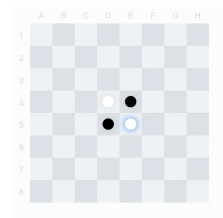
高级+白棋后手:

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✓	621s	白棋获胜, 领先棋子数: 32

确定

测试详情 隐藏棋盘



棋局胜负: 白棋赢
先后手: 白棋后手
棋局难度: 初级
当前棋子: 白棋
当前坐标: E5

确定

六、总结

- 本实验，我们实现了蒙特卡洛搜索数（并且是MinMax形式的），分别实现了四个过程：选择、扩展、模拟、反向传播。
- 实现了一个AI选手，并正反手教学，战胜了平台的ai。
- 在测试的时候，发现时间太长未必太好，但是太短了又导致解不够好。
- 发现其实策略十分重要，有时盲目的局部贪心策略可能还不如随机游走好，然后策略取决于对游戏的理解，比如在这个游戏中，对某个落子位置，如果它是有至少相邻两边的是同色棋或者边，那么这个落子位置更好。当然，四个角落的位置是最好的，边上的也很好。这些位置难以被翻转。